

SEGURANÇA NO PIPELINE

Integrar segurança desde o início do pipeline de DevSecOps é essencial para identificar vulnerabilidades, reduzir riscos e garantir a entrega contínua de software confiável.



CÓDIGO

Análise estática e boas práticas para identificar falhas cedo.



DEPENDÊNCIAS

Verificação de bibliotecas e componentes de terceiros.



INFRAESTRUTURA • APLICAÇÃO

Configurações seguras e testes para ambientes e aplicações.



DEVSECOPS APLICADO NA PRÁTICA
SONARQUBE • TRIVY • KICS • NUCLEI



DISCIPLINA: CLOUD SECURITY
• GRUPO 2



FONTES: README.MD • REPORTS/
EVIDENCIAS/ • LAB.MD

O QUE A TURMA FARÁ NOS PROXIMOS 30 MINUTOS



ESTRUTURA DA APRESENTAÇÃO

A SESSÃO SEGUE A LÓGICA CONCEITO → EXECUÇÃO → LEITURA CRÍTICA, PERMITINDO QUE CADA ETAPA CONSTRUA SOBRE A ANTERIOR. O OBJETIVO É QUE AO FINAL TODOS CONSIGAM INTERPRETAR UM RELATÓRIO DE SCANNER E DISTINGUIR ACHADOS BLOQUEANTES DE AVISOS INFORMATIVOS.



AGENDA E TEMPOS



-5 MIN

ABERTURA E CONTEXTO DO PROJETO



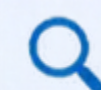
-8 MIN

APRESENTAÇÃO DAS QUATRO FERRAMENTAS



-12 MIN

LABORATÓRIO PRÁTICO GUIADO



-3 MIN

ANÁLISE FINAL E COMPARATIVO



-2 MIN

PERGUNTAS E ESCLARECIMENTOS



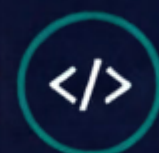


ONDE CADA ANÁLISE ENTRA NO PROJETO

Integramos a segurança em cada etapa do ciclo de desenvolvimento. As análises estática, dinâmica e de dependências acontecem de forma contínua e automatizada, garantindo que vulnerabilidades sejam detectadas cedo e corrigidas antes de impactar produção.



AS QUATRO ANÁLISES SE COMPLEMENTAM AO LONGO DO PIPELINE, COBRINDO DESDE O CÓDIGO-FONTE ATÉ O COMPORTAMENTO DA APLICAÇÃO EM EXECUÇÃO.



A MAIOR PARTE DAS FALHAS PODE SER IDENTIFICADA ANTES DA APLICAÇÃO CHEGAR À PRODUÇÃO. AINDA ASSIM, ALGUMAS SÓ APARECEM DURANTE A EXECUÇÃO, POR ISSO O PROJETO COMBINA ANÁLISES ANTECIPADAS COM TESTES EM RUNTIME.



SONARQUBE

COMO FERRAMENTA DE SAST, O SONARQUBE ANALISA O CÓDIGO-FONTE ANTES DA EXECUÇÃO DA APLICAÇÃO. NO PROJETO, O SCANNER VERIFICA APP/SERVER.PY E ENVIA OS RESULTADOS AO SERVIDOR SONARQUBE COMMUNITY, QUE CLASSIFICA OS ACHADOS POR REGRA, SEVERIDADE E LOCALIZAÇÃO. A CONFIGURAÇÃO DA ANÁLISE ESTÁ DEFINIDA EM SONAR-PROJECT.PROPERTIES, E AS EVIDÊNCIAS FORAM PRESERVADAS NOS RELATÓRIOS DO REPOSITÓRIO.



SONARQUBE: O QUE MUDOU

CORREÇÃO APLICADA E IMPACTO

Atualizamos a versão do Sonarqube e aplicamos as correções identificadas pela análise de segurança, resolvendo vulnerabilidades críticas e melhorando a proteção da plataforma.



ANTES DA CORREÇÃO

- 2 ISSUES ABERTAS
- 0 HOTSPOTS
- USO DE MD5 IDENTIFICADO
- CÓDIGO ANALISADO: HASHLIB_MD5()
- SITUAÇÃO INICIAL: VULNERÁVEL



DEPOIS DA CORREÇÃO

- MD5 SUBSTITUÍDO POR SHA-256
- 1 ISSUE RESOLVIDA
- 1 ISSUE AINDA ABERTA
- RISCO REMOVIDO NO CASO DO HASH
- HTTP PERMANECE COMO LIMITAÇÃO ACEITA NO LABORATÓRIO



A atualização e os ajustes realizados **reduzem o risco** da plataforma, **fortalecem o pipeline** de DevSecOps e garantem **conformidade continua** com as melhores práticas de segurança.



TRIVY: SCA NAS DEPENDÊNCIAS

Identificação de vulnerabilidades conhecidas nas dependências do projeto



O TRIVY REALIZA SCA ANALISANDO AS DEPENDÊNCIAS DECLARADAS EM REQUIREMENTS.TXT E COMPARANDO SUAS VERSÕES COM UMA BASE DE VULNERABILIDADES CONHECIDAS. NO CENÁRIO VULNERÁVEL, O PROJETO UTILIZAVA JINJA2 3.1.4 E FORAM IDENTIFICADAS TRÊS VULNERABILIDADES DE SEVERIDADE MEDIUM: CVE-2024-56201, CVE-2024-56326 E CVE-2025-27516. AS DUAS PRIMEIRAS POSSUEM CORREÇÃO A PARTIR DA VERSÃO 3.1.5, ENQUANTO A CVE-2025-27516 EXIGE JINJA2 3.1.6. POR ISSO, O CENÁRIO CORRIGIDO FOI ATUALIZADO PARA 3.1.6, ELIMINANDO OS TRÊS ACHADOS NAS DEPENDÊNCIAS DECLARADAS.\

OS RESULTADOS SÃO PRESERVADOS EM JSON E SARIF, E O PROCESSO TAMBÉM GERA UM SBOM CYCLONEDX PARA INVENTARIAR OS COMPONENTES ANALISADOS.



TRIVY: CORREÇÃO, INVENTÁRIO E LIMITES



CORREÇÃO APLICADA

JINJA2 3.1.4 → 3.1.6
([REQUIREMENTS.TXT](#))

AS TRÊS VULNERABILIDADES IDENTIFICADAS NO CENÁRIO INICIAL DEIXARAM DE APARECER APÓS A ATUALIZAÇÃO DA DEPENDÊNCIA.

CVE-2025-27516
CVE-2024-56201
CVE-2024-56326

RESULTADO: 0 VULNERABILIDADES DETECTADAS NO CONJUNTO DE DEPENDÊNCIAS DECLARADO.
GATE CI/CD: BLOQUEIO AUTOMÁTICO PARA ACHADOS HIGH/CRITICAL.



SBOM E LIMITES DO SCAN

O TRIVY TAMBÉM GERA UM SBOM EM CYCLONEDX, CRIANDO UM INVENTÁRIO DOS COMPONENTES ANALISADOS. ESSE ARTEFATO AJUDA NA RASTREABILIDADE DAS DEPENDÊNCIAS E PODE SER REUTILIZADO EM OUTRAS ETAPAS DO PIPELINE.

O RESULTADO, PORÉM, NÃO SIGNIFICA QUE TODA A APLICAÇÃO ESTÁ LIVRE DE VULNERABILIDADES: ESSE SCAN VERIFICA O CONJUNTO ANALISADO DE DEPENDÊNCIAS E NÃO SUBSTITUI SAST, IAC OU DAST. A PRÓPRIA ESTRUTURA DO TRABALHO EXIGE QUE ESSAS CATEGORIAS SEJAM TRATADAS COMO COMPLEMENTARES.





KICS: REVISAR A INFRAESTRUTURA ANTES DE IMPLANTAR



O KICS REALIZA ANÁLISE ESTÁTICA DO MANIFESTO KUBERNETES DEPLOYMENT.YAML ANTES DA IMPLANTAÇÃO, PROCURANDO CONFIGURAÇÕES INSEGURAS NA INFRAESTRUTURA COMO CÓDIGO. NO CENÁRIO VULNERÁVEL, FORAM IDENTIFICADOS 2 ACHADOS HIGH, 1 MEDIUM E 6 LOW. OS DOIS ACHADOS DE MAIOR SEVERIDADE ESTÃO RELACIONADOS A PRIVILÉGIOS EXCESSIVOS NO CONTAINER. OS RESULTADOS SÃO PRESERVADOS EM JSON E SARIF, PERMITINDO IDENTIFICAR A REGRA, A SEVERIDADE E A LOCALIZAÇÃO DE CADA PROBLEMA NO MANIFESTO.

COMO A ANÁLISE ACONTECE ANTES DO DEPLOY, O KICS PERMITE CORRIGIR CONFIGURAÇÕES INSEGURAS ANTES QUE ELAS CHEGUEM AO AMBIENTE DE EXECUÇÃO.

FONTES: FIXTURES/VULNERAVEL/DEPLOYMENT.YAML · REPORTS/KICS/VULNERAVEL/RESULTS.JSON · LAB.MD



KICS: OS DOIS HIGH QUE BLOQUEIAM O PROJETO

Dois achados HIGH impedem a aprovação do cenário vulnerável. Ambos estão relacionados a privilégios excessivos no container e são classificados pelo KICS como CWE-269. Após a correção das configurações, os achados bloqueantes deixam de aparecer



ACHADOS BLOQUEANTES

PRIVILEGED: TRUE
EXECUTA O CONTAINER EM MODO PRIVILEGIADO, CONCEDENDO CAPACIDADES EXCESSIVAS
.ALLOWPRIVILEGEESCALATION: TRUE
PERMITE QUE PROCESSOS ADQUIRAM PRIVILÉGIOS SUPERIORES AOS ORIGINALMENTE CONCEDIDOS.

AMBOS:

- SEVERIDADE HIGH;
- CWE-269;
- LOCALIZADOS NO SECURITYCONTEXT DE DEPLOYMENT.YAML.

RESULTADO DO CENÁRIO VULNERÁVEL: 2 HIGH → GATE BLOQUEADO.



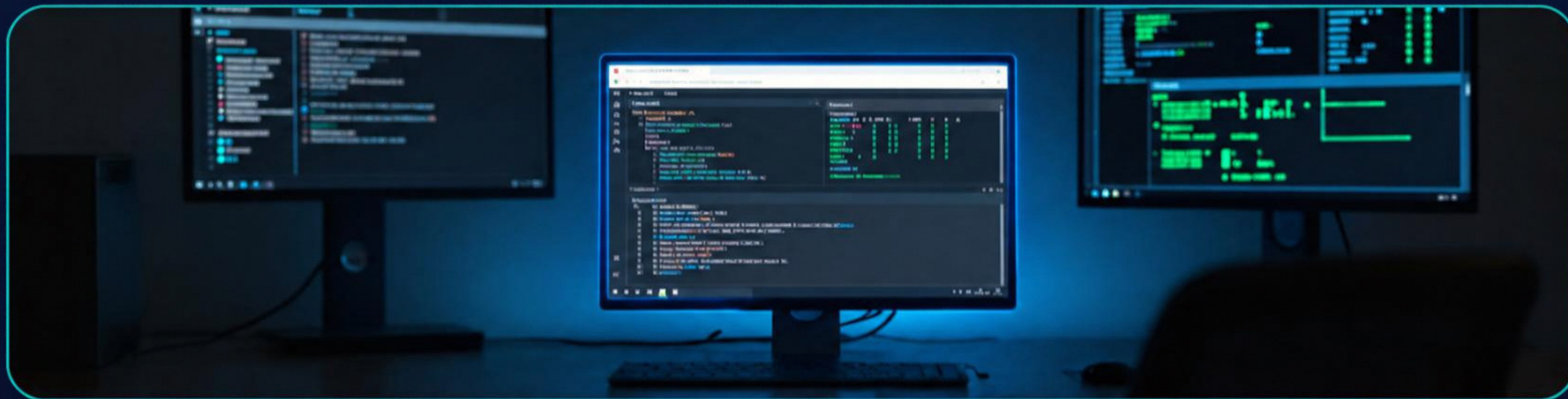
CORREÇÕES APLICADAS

PRIVILEGED: FALSE
REMOVE A EXECUÇÃO PRIVILEGIADA DO CONTAINER.
.ALLOWPRIVILEGEESCALATION: FALSE
IMPEDE A ESCALADA DE PRIVILÉGIOS.
READONLYROOTFILESYSTEM: TRUE
ADICIONA PROTEÇÃO AO SISTEMA DE ARQUIVOS RAIZ E REMOVE UM ACHADO LOW ADICIONAL.

APÓS A CORREÇÃO:

- 0 HIGH / CRITICAL;
- 1 MEDIUM E 5 LOW PERMANECEM;
- GATE APROVADO COM CÓDIGO 0.





✓ NUCLEI v3

NUCLEI: OBSERVAR O COMPORTAMENTO DA APLICAÇÃO



O DAST complementa SAST, SCA e IaC porque observa comportamentos que só existem em runtime.

O NUCLEI ANALISA A APLICAÇÃO ENQUANTO ELA ESTÁ EM EXECUÇÃO, ENVIANDO REQUISIÇÕES HTTP E AVALIANDO AS RESPOSTAS COM BASE EM TEMPLATES. NO PROJETO, UTILIZAMOS UM TEMPLATE PRÓPRIO PARA O ENDPOINT /DEBUG, CONFIGURADO PARA RECONHECER HTTP 200 E DOIS MARCADORES DE CONFIGURAÇÃO FICTÍCIA. NO CENÁRIO VULNERÁVEL, O TEMPLATE IDENTIFICA 1 CORRESPONDÊNCIA MEDIUM. APÓS A CORREÇÃO, O MESMO TESTE RETORNA 0 CORRESPONDÊNCIAS. TODA A EXECUÇÃO OCORRE EM AMBIENTE DOCKER LOCAL E CONTROLADO.



NUCLEI: RESULTADO E LIMITES DO TESTE

COMPARAÇÃO ENTRE O CENÁRIO VULNERÁVEL E O CORRIGIDO

O mesmo template foi executado nos dois cenários para verificar se a exposição presente no endpoint /debug continuava detectável após a correção. O resultado mostra a mudança de comportamento da aplicação sem alterar a lógica do teste.



ANTES DA CORREÇÃO

- ENDPOINT ANALISADO: /DEBUG
- REQUISIÇÃO: GET
- RESULTADO: 1 CORRESPONDÊNCIA
- SEVERIDADE: MEDIUM
- DADOS ENCONTRADOS: CONFIGURAÇÃO FICTÍCIA DE LABORATÓRIO
- TEMPO REGISTRADO NO RELATÓRIO: ~0,72 S



DEPOIS DA CORREÇÃO

- MESMO ENDPOINT E MESMO TEMPLATE
- RESULTADO: 0 CORRESPONDÊNCIAS
- A CONDIÇÃO PROCURADA DEIXOU DE SER DETECTADA
- NENHUM NOVO MATCHER FOI CRIADO PARA "FORÇAR" O RESULTADO
- TEMPO REGISTRADO NO RELATÓRIO: ~0,63 S



O resultado demonstra apenas o comportamento verificado por esse template específico. Um scan com um único template não representa uma auditoria DAST completa da aplicação.



Arquitetura do projeto DevSecOps

Grupo 2 • Aplicação Python • Cenários vulnerável e corrigido



GitHub • CP01-DevSecOps



Código + dependências + manifestos + evidências

Validação principal • Local e GitHub Actions

Push em main / Pull request / Execução manual



requirements.txt

Jinja2 + MarkupSafe



Trivy • SCA

Vulnerabilidades nas dependências

3 MEDIUM → 0 achados*



deployment.yaml

Manifesto Kubernetes



KICS • IaC

Configurações inseguras

2 HIGH → 0 HIGH/CRITICAL



Gate
HIGH / CRITICAL

Com bloqueantes

Sem bloqueantes



Pipeline reprovado



Pipeline aprovado



Relatórios JSON / SARIF + resumo do gate

Artefatos preservados no GitHub Actions

Erro operacional também

reprova a execução.

Análises complementares • Ambiente local



app/server.py



SonarScanner → SonarQube

SAST • Análise do código

Hash MD5 → SHA-256

Painel local: 127.0.0.1:19000

Docker Compose • Rede interna isolada



Nuclei • DAST

Template próprio

HTTP /debug



Aplicação Python :8080

Jinja2 • Vulnerável / Corrigida

Exposição em /debug: 1 achado → 0

Alvos sem portas publicadas.

SonarQube e Nuclei ficam fora do gate principal.



* Zero achados nas dependências declaradas, na análise registrada.

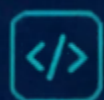
Manifestos analisados sem criar cluster Kubernetes. Gate aprovado não significa ausência de todos os riscos.

LABORATÓRIO: ABRIR O ROTEIRO E CONFERIR A PASTA

A PARTIR DAQUI COMEÇA O LABORATÓRIO PRÁTICO. VAMOS EXECUTAR TRIVY E KICS NOS CENÁRIOS VULNERÁVEL E CORRIGIDO, COMPARAR OS RESULTADOS E GERAR O COMPROVANTE FINAL. ANTES DE INICIAR, ABRA O LAB.MD E CONFIRME QUE O REPOSITÓRIO ESTÁ COMPLETO E QUE O DOCKER ESTÁ DISPONÍVEL NA MÁQUINA.



<https://github.com/MaduKa01/CP01-DevSecOps/blob/main/LAB.md>



PRÁTICA



SEGURANÇA



EVIDÊNCIAS



LABORATÓRIO: CONFERIR AS VERSÕES

ANTES DE INICIAR AS ANÁLISES, VAMOS CONFIRMAR SE TODOS ESTÃO UTILIZANDO AS MESMAS VERSÕES DAS FERRAMENTAS. COMO AS IMAGENS ESTÃO FIXADAS NO DOCKER-COMPOSE.YML, ESSA VERIFICAÇÃO AJUDA A GARANTIR QUE OS RESULTADOS DO LABORATÓRIO SEJAM REPRODUZÍVEIS ENTRE AS MÁQUINAS.

</> COMANDOS – TRIVY

```
docker compose run --rm trivy --version
```

Resultado esperado:

Version: 0.74.0

Também vale observar a data da base de vulnerabilidades, pois ela pode mudar mesmo quando a versão do scanner permanece a mesma.

</> COMANDOS – KICS

```
docker compose run --rm kics
```

version Kics v2.1.0

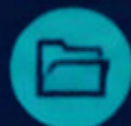
```
docker compose run --rm kics version
```

Resultado esperado:

2.1.20



MESMAS VERSÕES + MESMO AMBIENTE DOCKER = RESULTADOS MAIS COMPARÁVEIS E REPRODUZÍVEIS ENTRE AS MÁQUINAS DO GRUPO

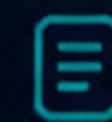
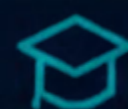


TRIVY: SCAN INICIAL

Execução do scan e observação dos achados antes de qualquer correção.



EXECUTE O TRIVY SOBRE O ARQUIVO FIXTURES/VULNERAVEL/REQUIREMENTS.TXT PARA OBSERVAR O ESTADO INICIAL DAS DEPENDÊNCIAS ANTES DE QUALQUER CORREÇÃO. O PROJETO UTILIZA JINJA2 3.1.4 E O SCAN IDENTIFICA TRÊS VULNERABILIDADES DE SEVERIDADE MEDIUM: CVE-2025-27516, CVE-2024-56201 E CVE-2024-56326. ESSE RESULTADO COMPROVA QUE A VERSÃO DECLARADA POSSUI VULNERABILIDADES CONHECIDAS, MAS NÃO DEMONSTRA QUE ELAS FORAM EXPLORADAS NA APLICAÇÃO. COMO O GATE DO PROJETO BLOQUEIA APENAS ACHADOS HIGH/CRITICAL, ESSES TRÊS MEDIUM SÃO REPORTADOS, MAS NÃO INTERROMPEM O PIPELINE.





LABORATÓRIO: INTERPRETAR UMA CVE

A CVE-2025-27516 AFETA O JINJA2 3.1.4 E ESTÁ ASSOCIADA AO CWE-1336. A VULNERABILIDADE PERMITE CONTORNAR A SANDBOX EM DETERMINADAS CONDIÇÕES QUANDO O USUÁRIO CONTROLA O CONTEÚDO DE UM TEMPLATE. NO NOSSO PROJETO, O ENDPOINT /PREVIEW RECEBE TEMPLATES ENVIADOS PELO USUÁRIO E OS PROCESSA COM SANDBOXEDENVIRONMENT, TORNANDO O CENÁRIO TECNICAMENTE RELEVANTE.

O TRIVY IDENTIFICOU CORRETAMENTE A DEPENDÊNCIA VULNERÁVEL E INDICOU A CORREÇÃO PARA JINJA2 3.1.6. PORÉM, NÃO EXECUTAMOS UMA EXPLORAÇÃO PARA DEMONSTRAR FUGA DA SANDBOX. PORTANTO, CLASSIFICAMOS O ACHADO COMO VERDADEIRO POSITIVO PARA A DEPENDÊNCIA AFETADA, SEM AFIRMAR COMPROMETIMENTO DA APLICAÇÃO.

CVE-2025-27516 · CWE-1336 · JINJA2 3.1.4 → CORREÇÃO 3.1.6



LABORATÓRIO: EXECUTAR O KICS VULNERÁVEL



O KICS ANALISA O MANIFESTO VULNERÁVEL DEPLOYMENT.YAML ANTES DE QUALQUER CORREÇÃO. A EXECUÇÃO IDENTIFICA 2 ACHADOS HIGH, 1 MEDIUM E 6 LOW. OS DOIS ACHADOS BLOQUEANTES ESTÃO RELACIONADOS A PRIVILÉGIOS EXCESSIVOS NO CONTAINER: PRIVILEGED: TRUE E ALLOWPRIVILEGEESCALATION: TRUE. O PROCESSO RETORNA CÓDIGO 50, INDICANDO QUE A POLÍTICA DE SEGURANÇA FOI VIOLADA E QUE O GATE DEVE BLOQUEAR O CENÁRIO VULNERÁVEL.



KICS: ENTENDER OS DOIS ACHADOS HIGH

POR QUE ESSAS CONFIGURAÇÕES REPRESENTAM RISCO?

Os dois achados bloqueantes estão relacionados ao excesso de privilégios concedidos ao container. Embora sejam diferentes, ambos pertencem ao CWE-269 e aumentam o impacto de uma possível exploração.



ANTES DA CORREÇÃO

PRIVILEGED: TRUE
EXECUTA O CONTAINER EM MODO PRIVILEGIADO,
CONCEDENDO CAPACIDADES MUITO SUPERIORES ÀS
NECESSÁRIAS.
ALLOWPRIVILEGEESCALATION: TRUE
PERMITE QUE UM PROCESSO DENTRO DO CONTAINER
ADQUIRA PRIVILÉGIOS SUPERIORES AOS ORIGINALMENTE
CONCEDIDOS.
CLASSIFICAÇÃO: HIGH
CWE: CWE-269
RESULTADO: 2 ACHADOS BLOQUEANTES



DEPOIS DA CORREÇÃO

PRIVILEGED: FALSE
REMOVE A EXECUÇÃO PRIVILEGIADA.
ALLOWPRIVILEGEESCALATION: FALSE
IMPEDE A ESCALADA DE PRIVILÉGIOS.
READONLYROOTFILESYSTEM: TRUE
TORNA O SISTEMA DE ARQUIVOS RAIZ SOMENTE LEITURA
E ELIMINA UM ACHADO LOW ADICIONAL.
APÓS A CORREÇÃO:
0 HIGH / CRITICAL
1 MEDIUM + 5 LOW PERMANECEM



A correção remove os achados que violavam a política do gate, mas não significa que todos os avisos desapareceram: o cenário corrigido ainda mantém achados de menor severidade.

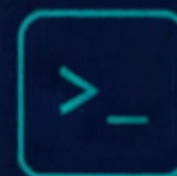




LABORATÓRIO: EXECUTAR O CENÁRIO CORRIGIDO

Agora repetimos as análises sobre o estado corrigido, mantendo a mesma política de segurança. A diferença está nos artefatos: Jinja2 foi atualizado para 3.1.6 e o manifesto Kubernetes teve os privilégios excessivos removidos.

			
DEPENDÊNCIA	PRIVILÉGIOS	ESCALADA	FILESYSTEM
Jinja2 3.1.4 → 3.1.6	privileged: false	allowPrivilegeEscalation: false	readOnlyRootFilesystem: true



Execute os três comandos da etapa 4 do LAB.md: Trivy corrigido → gate Trivy → KICS corrigido.



RESULTADO ESPERADO: TRIVY SEM VULNERABILIDADES NAS DEPENDÊNCIAS DECLARADAS; KICS COM 0 HIGH/CRITICAL, PERMANECENDO 1 MEDIUM E 5 LOW. OS DOIS GATES RETORNAM CÓDIGO 0.
A POLÍTICA NÃO FOI ALTERADA PARA OBTER O RESULTADO VERDE: OS MESMOS CRITÉRIOS FORAM APLICADOS AOS ARTEFATOS CORRIGIDOS.

LABORATÓRIO: O QUE AINDA PERMANECE

PIPELINE VERDE NÃO SIGNIFICA AUSÊNCIA TOTAL DE AVISOS.

Após as correções, os achados que violavam a política do gate foram removidos. Isso não significa que todos os alertas desapareceram: o resultado verde indica apenas que o cenário corrigido atende ao limite definido para HIGH e CRITICAL.



TRIVY DEPENDÊNCIAS CORRIGIDAS

JINJA2 ATUALIZADO PARA 3.1.6
RESULTADO DO SCAN:
0 VULNERABILIDADES DETECTADAS NO
CONJUNTO DE DEPENDÊNCIAS DECLARADO.
GATE: APROVADO
CÓDIGO DE SAÍDA: 0

O resultado se refere às dependências analisadas nesse cenário, não à segurança completa da aplicação.



KICS AVISOS REMANESCENTES

APÓS REMOVER OS DOIS HIGH:
0 HIGH
0 CRITICAL
1 MEDIUM
5 LOW
GATE: APROVADO
CÓDIGO DE SAÍDA: 0

Os avisos restantes permanecem visíveis nos relatórios e não foram ocultados para produzir um resultado verde.



“Verde” significa conformidade com a política definida — não ausência absoluta de riscos ou oportunidades de melhoria.



Pipeline vermelho — **FAILURE**

✓ ✗ Executar SCA e IaC com gate HIGH/CRITICAL

```
1 ▶ Run python3 scripts/run_scan.py "$LAB_VARIANT"
6 vulneravel: FAIL | bloqueantes HIGH/CRITICAL: 2
7 Trivy: {'MEDIUM': 3} KICS: {'CRITICAL': 0, 'HIGH': 2, 'INFO': 0, 'LOW': 6, 'MEDIUM': 1, 'TRACE': 0}
8 Evidências: reports/gate/vulneravel
9 Error: Process completed with exit code 1.
```

Pipeline verde — **SUCCESS**

✓ ✓ Executar SCA e IaC com gate HIGH/CRITICAL

```
1 ▶ Run python3 scripts/run_scan.py "$LAB_VARIANT"
6 corrigido: PASS | bloqueantes HIGH/CRITICAL: 0
7 Trivy: {} KICS: {'CRITICAL': 0, 'HIGH': 0, 'INFO': 0, 'LOW': 5, 'MEDIUM': 1, 'TRACE': 0}
8 Evidências: reports/gate/corrigido
```

• FAILURE — RUN 36167909434
• SUCCESS — RUN 36167707788

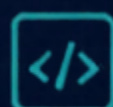
BUILDS SEGURAS EM DEVSECOPS

PIPELINE: DO VERMELHO AO VERDE

Mesma política de segurança, resultados diferentes após a correção dos artefatos.



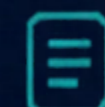
O MESMO COMMIT E A MESMA POLÍTICA DE GATE FORAM APLICADOS AOS DOIS CENÁRIOS. NO VULNERÁVEL, O KICS IDENTIFICOU 2 ACHADOS HIGH E O PIPELINE FOI BLOQUEADO. APÓS A CORREÇÃO DAS CONFIGURAÇÕES, OS ACHADOS BLOQUEANTES FORAM REMOVIDOS E O MESMO GATE PASSOU COM CÓDIGO 0. O RESULTADO VERDE FOI OBTIDO PELA CORREÇÃO DOS ARTEFATOS — NÃO PELA REDUÇÃO DOS CRITÉRIOS DE SEGURANÇA.



FONTE DO PIPELINE
GITHUB ACTIONS RUNS



POLÍTICA HIGH/CRITICAL
BLOQUEIAM



EVIDÊNCIAS
JSON · SARIF · ARTIFACTS
PRESERVADOS



COMPROVANTE E DUAS PERGUNTAS

O comprovante confirma que o laboratório foi executado com relatórios atuais e registra os resultados observados por cada participante.



DOCKER COMPOSE RUN --RM VERIFY PYTHON SCRIPTS/COMPROVANTE.PY

O SCRIPT VALIDA OS RELATÓRIOS RECENTES E GERA O COMPROVANTE INDIVIDUAL DA EXECUÇÃO.

1

TRIVY: QUAL TIMESTAMP APARECE NO SEU RELATÓRIO E QUAL VERSÃO DO JINJA2 CORRIGE A VULNERABILIDADE ANALISADA?

2

KICS: QUAL TIMESTAMP APARECE NO SEU RELATÓRIO, QUANTOS ACHADOS HIGH FORAM ENCONTRADOS NO CENÁRIO VULNERÁVEL E QUAIS PROPRIEDADES FORAM CORRIGIDAS?



FONTES: LAB.MD · SCRIPTS/COMPROVANTE.PY · RELATÓRIOS TRIVY E KICS



AS FERRAMENTAS COBREM PARTES DIFERENTES DO PROBLEMA



TOOLCHAIN COMBINADA

CADA FERRAMENTA ANALISA UMA SUPERFÍCIE DIFERENTE DO PROJETO E PRODUZ EVIDÊNCIAS COMPLEMENTARES. SAST VERIFICA O CÓDIGO PRÓPRIO, SCA ANALISA DEPENDÊNCIAS, IAC SECURITY REvisa CONFIGURAÇÕES DE INFRAESTRUTURA E DAST OBSERVA A APLICAÇÃO EM EXECUÇÃO. NENHUMA DESSAS CATEGORIAS SUBSTITUI AS DEMAIS.



COMPARATIVO: FERRAMENTA x CATEGORIA

	SONARQUBE	SAST → APP/SERVER.PY → ISSUES E QUALIDADE → NÃO ANALISA DEPENDÊNCIAS NEM RUNTIME
	GATE	CI/CD → USA RESULTADOS DO TRIVY + KICS → BLOQUEIA HIGH/CRITICAL → RETORNA PASS/FAIL
	TRIVY	SCA → REQUIREMENTS.TXT → CVES, JSON, SARIF E SBOM → NÃO COMPROVA EXPLORAÇÃO
	KICS	IAC SECURITY → DEPLOYMENT.YAML → ACHADOS HIGH/MEDIUM/LOW → NÃO EXECUTA O CLUSTER
	NUCLEI	DAST → APLICAÇÃO HTTP /DEBUG → MATCHES POR TEMPLATE → REQUER RUNTIME E TEM COBERTURA LIMITADA AO TEMPLATE USADO



A cobertura aumenta pela combinação das quatro categorias: código, dependências, infraestrutura e comportamento em runtime.



DUAS MÁQUINAS, MESMOS RESULTADOS PRINCIPAIS

O laboratório foi validado em dois ambientes diferentes: Windows com WSL2 e macOS Apple Silicon. Apesar das diferenças entre os sistemas, os resultados principais permaneceram consistentes: o Trivy encontrou as mesmas 3 vulnerabilidades MEDIUM e o KICS os mesmos 2 achados HIGH no cenário vulnerável.

Após as correções, os achados bloqueantes desapareceram nos dois ambientes e os testes automatizados foram concluídos com sucesso.



A execução ocorreu sob condições controladas, com mesma imagem base e versões de ferramentas, garantindo que diferenças entre plataformas não interferem nos resultados de segurança. O pipeline DevSecOps se mantém estável e confiável em ambos os sistemas operacionais.

TESTES EXECUTADOS EM MAC (APPLE SILICON) E WINDOWS COM WSL2, USANDO OS MESMOS CONTAINERS DOCKER FIXADOS POR VERSÃO. OS ACHADOS PRINCIPAIS DO TRIVY E DO KICS FORAM IDENTÍFICOS NOS DOIS AMBIENTES, CONFIRMANDO A REPETIBILIDADE DAS EVIDÊNCIAS. OS TESTES DE GATE PRODUZIRIAM O MESMO CÓDIGO DE SAÍDA (EXIT NO CENÁRIO VULNERÁVEL, EXIT NO CORRIGIDO). VARIÁVEIS DE TEMPO DE EXECUÇÃO FORAM OBSERVADAS, MAS NÃO AFETARAM OS RESULTADOS. A CONSISTÊNCIA ENTRE REPORÇÃO QUE OS RELATÓRIOS REFLETEM O COMPORTAMENTO REAL DAS FERRAMENTAS, INDEPENDENTE DO SISTEMA OPERACIONAL DO ATALHADOR.



LIMITAÇÕES QUE OS RELATÓRIOS NÃO RESOLVEM



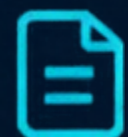
O QUE OS RELATÓRIOS NÃO COMPROVAM

- NÃO DEMONSTRAMOS EXPLORAÇÃO REAL DAS VULNERABILIDADES SCA.
- O KICS ANALISOU O MANIFESTO; NENHUM CLUSTER KUBERNETES FOI CRIADO.
- O NUCLEI UTILIZOU APENAS 1 TEMPLATE CUSTOMIZADO.
- O HTTP PERMANECE COMO LIMITAÇÃO CONHECIDA DO LABORATÓRIO.
- 0/3 FALSOS POSITIVOS REFERE-SE SOMENTE AOS TRÊS ACHADOS REVISADOS.
- OS RESULTADOS NÃO REPRESENTAM COBERTURA COMPLETA DA APLICAÇÃO.



INTERPRETAÇÃO CONTEXTUAL NECESSÁRIA

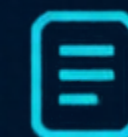
- DETECÇÃO NÃO SIGNIFICA EXPLORAÇÃO COMPROVADA.
- UM GATE VERDE INDICA CONFORMIDADE COM A POLÍTICA HIGH/CRITICAL, NÃO AUSÊNCIA TOTAL DE RISCO.
- ACHADOS MEDIUM E LOW PERMANECEM NO CENÁRIO CORRIGIDO.
- MÉTRICAS AUTOMÁTICAS NÃO SUBSTITUEM O ENSAIO HUMANO DO LABORATÓRIO.
- OS RESULTADOS VALEM PARA OS ARTEFATOS, VERSÕES E CONDIÇÕES TESTADAS.
- NÃO É CORRETO AFIRMAR “100% SEGURO” OU “ZERO VULNERABILIDADES”.



DEVSECOPS APLICADO NA PRÁTICA
SONARQUEE • TRIVVY • KICS • NUCLEI



DISCIPLINA: CLOUD SECURITY
• GRUPO 2



FONTES: README.MD • REPORTS/
EVIDENCIAS/ • LAB.MD





ENTREGA, COLABORAÇÃO E RASTREABILIDADE

O REPOSITÓRIO CONCENTRA CÓDIGO, CONFIGURAÇÕES, RELATÓRIOS E EVIDÊNCIAS PRODUZIDAS DURANTE O TRABALHO, PERMITINDO RASTREAR TANTO A EXECUÇÃO TÉCNICA QUANTO A COLABORAÇÃO DO GRUPO. AS CONTRIBUIÇÕES FORAM REGISTRADAS EM BRANCHES, COMMITS E PULL REQUESTS, ENQUANTO OS RESULTADOS DAS FERRAMENTAS PERMANECERAM VERSIONADOS EM FORMATOS COMO JSON E SARIF.

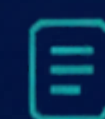
A ENTREGA REÚNE AINDA O LAB.MD, SCRIPTS DE VALIDAÇÃO, COMPROVANTES DE EXECUÇÃO E DEMAIS EVIDÊNCIAS UTILIZADAS NA APRESENTAÇÃO. O USO DE IA GENERATIVA TAMBÉM FOI DECLARADO EM USO-DE-IA.MD, CONFORME EXIGIDO PELO PROFESSOR, MANTENDO TRANSPARÊNCIA SOBRE O PROCESSO DE PRODUÇÃO E VALIDAÇÃO DO CONTEÚDO.



COLABORAÇÃO
BRANCHES · COMMITS ·
PULL REQUESTS



EVIDÊNCIAS
JSON · SARIF · COMPROVANTES



TRANSPARÊNCIA
USO-DE-IA.MD

FONTES E MATERIAL PARA CONTINUAR

O REPOSITÓRIO REÚNE O LAB.MD, RELATÓRIOS, EVIDÊNCIAS E OS ARTEFATOS UTILIZADOS DURANTE A APRESENTAÇÃO. PARA APROFUNDAR OS CONCEITOS E CONSULTAR CADA FERRAMENTA, O TRABALHO TAMBÉM REFERENCIA A DOCUMENTAÇÃO OFICIAL DO SONARQUBE, TRIVY, KICS E NUCLEI.

A PROPOSTA DA TOOLCHAIN NÃO É PROMETER SEGURANÇA ABSOLUTA, MAS REDUZIR RISCOS POR MEIO DE ANÁLISES COMPLEMENTARES, EVIDÊNCIAS REPRODUZÍVEIS E LIMITAÇÕES CLARAMENTE DOCUMENTADAS.

REPOSITÓRIO

Código · LAB.md ·
relatórios · evidências

DOCUMENTAÇÃO OFICIAL

SonarQube · Trivy ·
KICS · Nuclei

MATERIAL DO TRABALHO

Pesquisa final ·
README · guia técnico





PERGUNTAS?



ONDE TERMINA A EVIDÊNCIA DE CADA FERRAMENTA?
POR QUE O PIPELINE VERDE AINDA MANTÉM AVISOS?
COMO CONFIRMAR QUE O RELATÓRIO PERTENCE À EXECUÇÃO ATUAL?

RESPOSTAS BASEADAS NOS RELATÓRIOS, EVIDÊNCIAS VERSIONADAS E LAB.MD.